

Process Synchronization

1. The Problem: Race Conditions

When two or more processes (or threads) access and modify **shared data** concurrently, and the final result depends on the precise timing/interleaving of their execution, we have a **race condition**.

Classic example: two processes both do `count = count + 1` / `count = count - 1` on a shared variable `count = 5`.

```
{ count = count + 1; }      { count = count - 1; }
I.  Load R1, count      I.  Load R2, count
II. Inc R1                II. Dcr R2
III. Store count, R1      III. Store count, R2
```

Each high-level statement is actually **three machine instructions**. If they interleave badly — e.g., P1 loads count (5) into R1, then P2 loads count (5) into R2 before P1 stores back — one of the two updates is silently lost. The final value of `count` depends on the exact interleaving, which is exactly the definition of a race condition.

The shared resource being fought over (a variable, a buffer, a file) is called the **critical section** / **critical region** — the piece of code that accesses shared data and must not be executed by more than one process at a time.

2. Requirements for a Correct Critical-Section Solution

Any valid solution to the critical-section problem must guarantee:

1. **Mutual exclusion** — no two processes may be inside their critical sections at the same time.
 2. **Progress** — no assumptions about the number of CPUs or their relative speed; a process outside its critical section must not block others from entering theirs.
 3. **Bounded waiting** — no process executing in a non-critical region should be able to prevent another process from eventually entering its critical region.
 4. **No starvation** — no process should wait forever to enter its critical region.
-

3. Solution Categories

Level	Approach	Executes in
Hardware	Disabling/enabling interrupts, TSL (Test-and-Set-Lock)	Processor level
OS	Semaphore, Monitor	Kernel mode
Software	Lock variable, Strict alternation, Peterson's algorithm	User mode (non-privileged instructions)

3.1 Software solution: Lock variable

```
lock = 0;

P1:                                P2:
while (lock == 1);                while (lock == 1);
lock = 1;                          lock = 1;
// critical section              // critical section
lock = 0;                          lock = 0;
```

Flaw: the check (`while (lock==1)`) and the set (`lock=1`) are two separate, non-atomic steps. If P1 is preempted *right after* passing the while-check but *before* setting `lock=1` , P2 can also pass its check and set `lock=1` — now **both** processes are inside the critical section simultaneously. Mutual exclusion is violated.

3.2 Strict Alternation

```
turn = 1;

P1:                                P2:
while (turn == 2);                while (turn == 1);
// critical region              // critical region
turn = 2;                          turn = 1;
```

Flaw: violates **progress**. This *forces* strict turn-taking — if P1 finishes and doesn't need the critical section again for a while, but P2 does, P2 must still wait for `turn` to become 2 even though P1 isn't using (or wanting) the critical section. A process not interested in the critical section can still block one that is.

3.3 Peterson's Algorithm

Combines the lock-variable and turn ideas to fix both flaws above:

```
turn = 0;
interest[2] = {false, false};

P0:                                P1:
interest[0] = true;                interest[1] = true;
turn = 1;                          turn = 0;
while (turn == 1 && interest[1]==true); while (turn == 0 && interest[0]==true);
// critical section              // critical section
interest[0] = false;                interest[1] = false;
```

- `interest[i]` declares that process *i* **wants** to enter.
- `turn` resolves the case where **both** declare interest at the same time — whichever process set `turn` *last* politely yields to the other.
- Satisfies mutual exclusion, progress, and bounded waiting — but only works for **2 processes**, and relies on instructions executing without arbitrary hardware reordering (in reality, modern CPUs and compilers can reorder memory operations, so Peterson's algorithm technically needs memory barriers to be correct on real hardware — a subtlety worth mentioning if pressed, but the logical algorithm above is what's typically asked).

Interview soundbite: "Peterson's algorithm fixes the lock variable's atomicity problem and strict alternation's forced-turn-taking problem by combining an interest flag per process (so an uninterested process never blocks another) with a turn variable (so simultaneous interest is resolved deterministically)."

4. Semaphores (OS-level solution)

A semaphore is deceptively simple: it's **just an integer variable**, but access to it is restricted to two special operations — `wait()` (also called P, from Dutch *proberen*) and `signal()` (also called V, from *verhogen*) — both of which are **atomic**: while one process is inside `wait()` or `signal()`, no other process can be inside either operation on the same semaphore.

```
void wait(semaphore s) {
    s--;
    if (s < 0)
        block the process and place it in the semaphore's queue;
}

void signal(semaphore s) {
    s++;
    if (s <= 0)
        remove one process from the semaphore's queue and move it to ready;
}
```

- `wait(s)` — "asking for the key."
- `signal(s)` — "returning the key."

Types of semaphores

Type	Values	Use
Counting semaphore	Any integer ($-\infty$ to $+\infty$ conceptually)	Managing a pool of <i>N</i> identical resources
Binary semaphore (mutex)	Only 0 or 1	Implementing mutual exclusion (only one process in the critical section)

Achieving strict alternation correctly, with semaphores

```
semaphore s1 = 1, s2 = 0;

P1                                P2
while(1) {                        while(1) {
    wait(s1);                      wait(s2);
    printf("0");                  printf("1");
    signal(s2);                  signal(s1);
}
```

This reliably produces `010101...` — unlike the flawed software attempts above, because `wait` / `signal` are atomic and the OS itself manages the blocking/waking.

Interview soundbite: "A semaphore is just an integer, but the OS guarantees `wait` and `signal` are atomic — that atomicity is the entire reason semaphores succeed where naive software solutions fail."

5. Classic Synchronization Problems

5.1 Producer–Consumer Problem

- **Producer** creates data and places it in a shared **buffer**; **consumer** removes and uses it.
- If the buffer is **full**, the producer must block. If the buffer is **empty**, the consumer must block.
- The buffer itself is the critical section.

Solution (bounded buffer of size n):

```
semaphore mutex = 1;    // binary – protects the buffer itself
semaphore empty = n;    // counts empty slots
semaphore filled = 0;   // counts filled slots

Producer:                Consumer:
wait(empty);             wait(filled);
wait(mutex);             wait(mutex);
    insert_item(buffer);    remove_item(buffer);
signal(mutex);           signal(mutex);
signal(filled);          signal(empty);
```

- `empty` / `filled` are **counting** semaphores tracking how many slots are available/occupied.
- `mutex` is a **binary** semaphore ensuring only one process touches the buffer's internal pointers at a time.
- **Order matters:** always `wait(empty/filled)` *before* `wait(mutex)` — doing it the other way risks deadlock (e.g., a producer holding `mutex` while blocked on a full buffer, permanently locking the consumer out of `mutex` too).

5.2 Reader–Writer Problem

- **Multiple readers** can safely read shared data *simultaneously* (reading doesn't modify anything).
- A **writer** needs **exclusive** access — no other reader or writer may touch the data while it's writing.

Solution:

```
mutex m1 = 1, m2 = 1;
int count = 0;           // number of active readers

Writer:                  Reader:
wait(m1);                wait(m2);
// writing                count++;
signal(m1);              if (count == 1) wait(m1); // first reader locks out writers
                          signal(m2);
                          // reading
                          wait(m2);
                          count--;
                          if (count == 0) signal(m1); // last reader lets writers back in
                          signal(m2);
```

- **m2** protects the **count** variable itself (so two readers updating **count** don't race).
- **m1** is the actual writer-exclusion lock. Only the **first** reader to arrive acquires it (locking out writers); only the **last** reader to leave releases it.
- **Note:** this "readers-preference" version can starve writers if readers keep arriving continuously — a fair or writer-preference variant would need additional logic.

Reader-writer vs producer-consumer — the core distinction:

	Reader-Writer	Producer-Consumer
Concern	Access control (who can touch shared data)	Data flow through a buffer
Shared resource	Same data	A buffer
Concurrency allowed	Multiple readers together	Buffer-slot based (empty/filled counts)
Main issue	Read-write conflict	Full/empty buffer

5.3 Dining Philosophers Problem

- 5 philosophers sit around a table; each has **one chopstick** to their left and shares it with their right-hand neighbor. Eating requires **both** chopsticks.
- States: **Thinking** → **Hungry** → **Eating**.

Naive solution (deadlock-prone):

```
semaphore chopstick[5] = {1,1,1,1,1};

Philosopher(i):
while (true) {
    think();
    wait(chopstick[i]);           // pick left
    wait(chopstick[(i+1) % 5]);  // pick right
    eat();
    signal(chopstick[i]);
    signal(chopstick[(i+1) % 5]);
}
```

Deadlock scenario: if all 5 philosophers simultaneously pick up their *left* chopstick, every chopstick is now held, and every philosopher is stuck waiting forever for their right chopstick — a **circular wait**, one of the classic deadlock conditions (covered in the Deadlocks topic).

Common fixes (worth knowing exist, even if not implemented in full):

- Allow only 4 philosophers to sit/try at once (breaks the circular symmetry).
- Make one philosopher pick up chopsticks in the opposite order (right-then-left) — breaks the symmetric circular wait.
- Use a global resource-ordering / arbitrator approach.

Starvation vs deadlock here: deadlock = everyone stuck forever, simultaneously. Starvation = a specific philosopher is repeatedly passed over even though the *system* isn't stuck (others keep eating), because scheduling keeps favoring their neighbors.

6. Priority Inversion (and Priority Inheritance)

A subtle scheduling + synchronization interaction: a lower-priority process can end up indirectly blocking a higher-priority one via a shared lock.

Setup: three processes — **L** (low priority), **M** (medium), **H** (high). Arrival order: L, M, H.

Sequence of events:

1. L acquires a mutex and enters its critical section.
2. H arrives, tries to acquire the same mutex, and is blocked — waiting for L to release it.
3. M becomes ready and, since M has higher priority than L, **preempts L** (M doesn't need the mutex at all).
4. M runs to completion.
5. Only *now* does L resume, finish its critical section, and release the mutex.
6. H finally acquires the mutex and runs.

Actual execution order: **M → H → L**. But by priority, we wanted **H → M → L**. Because H was indirectly blocked by L, and L got preempted by the unrelated M, effectively **H waited behind M — a lower-priority process ran before a higher-priority one got to make progress**. This is **priority inversion**: L and H's priorities are *effectively* swapped for the duration L holds the mutex.

The fix: Priority Inheritance

When a low-priority process (L) is blocking a high-priority process (H) via a shared resource, **temporarily boost L's priority to H's priority** for as long as L holds that resource.

With inheritance: L (temporarily elevated to H's priority) can no longer be preempted by M. L finishes its critical section quickly, releases the mutex, drops back to its original low priority, and H immediately acquires the mutex and runs. Execution order becomes the desired **H → M → L** (or at least, M no longer improperly delays H).

Interview soundbite: "Priority inversion happens when a low-priority process holding a lock is preempted by a medium-priority process, indirectly delaying a high-priority process that's waiting on that same lock. Priority inheritance fixes it by temporarily raising the lock-holder's priority to match the highest-priority process waiting on it, so it can't be preempted by anything in between."

Interview Questions With Answers

Q1. What is a race condition, and why does it occur even for something as simple as

`count++` ?

Answer: A race condition occurs when the outcome of concurrent execution depends on the specific timing/interleaving of operations on shared data. `count++` looks atomic in source code but compiles to multiple machine instructions (load, increment, store); if two processes interleave those instructions, one process's update can be silently overwritten by the other's, because each pulled a stale copy of `count` before either wrote back.

Q2. List the four requirements a correct critical-section solution must satisfy.

Answer: Mutual exclusion (no two processes in their critical section simultaneously), progress (a process not in its critical section can't block others from entering theirs, and this decision can't be postponed indefinitely), bounded waiting (a limit on how many times other processes can enter before a waiting process gets its turn), and no starvation (no process waits forever).

Q3. Why does the simple lock-variable solution fail to guarantee mutual exclusion?

Answer: Because checking the lock (`while(lock==1)`) and setting it (`lock=1`) are two separate, non-atomic instructions. A process can be preempted in the gap between passing the check and setting the lock, letting another process also pass the check — both end up inside the critical section.

Q4. Why does strict alternation violate the "progress" requirement?

Answer: It forces processes to take turns via a shared `turn` variable regardless of whether the process whose turn it is actually wants to enter the critical section. If P1 doesn't need the critical section right now but it's still "its turn," P2 is blocked from entering even though the critical section is free — a process not interested in the critical section is still preventing another from proceeding.

Q5. How does Peterson's algorithm fix the problems in both the lock-variable and strict-alternation approaches?

Answer: It uses two mechanisms together: an `interest[i]` flag per process (so a process not wanting to enter never blocks the other, fixing strict alternation's flaw) and a shared `turn` variable used only to break ties when both processes declare interest simultaneously (fixing the lock variable's atomicity gap, since whoever sets `turn` last yields).

Q6. What is a semaphore, and why does it succeed where naive software solutions fail?

Answer: A semaphore is an ordinary integer variable, but the OS guarantees that its two allowed operations — `wait()` and `signal()` — are executed **atomically**: no other process can be inside either operation on that semaphore while one is in progress. That atomicity guarantee is exactly what the lock-variable software solution was missing, which is why semaphores reliably enforce mutual exclusion where naive attempts don't.

Q7. What's the difference between a counting semaphore and a binary semaphore?

Answer: A counting semaphore can take any integer value and is used to manage a pool of N identical resources (e.g., `empty` / `filled` counts in producer-consumer). A binary semaphore (mutex) only takes values 0 or 1 and is used specifically to enforce mutual exclusion — only one process can be "in" at a time.

Q8. In the producer-consumer solution, why must `wait(empty)` / `wait(filled)` happen before `wait(mutex)`, and not after?

Answer: If a process acquired `mutex` first and then blocked on `wait(empty)` or `wait(filled)` (because the buffer was full/empty), it would be holding `mutex` while blocked — permanently preventing the *other* process from ever acquiring `mutex` to change the buffer's state (which is exactly what would unblock the first process). That's a deadlock. Acquiring the buffer-slot semaphore first ensures a process only takes `mutex` when it's actually guaranteed to be able to proceed.

Q9. What is the fundamental difference between the reader-writer and producer-consumer problems?

Answer: Producer-consumer is about coordinating data *flowing through* a bounded buffer (full/empty conditions). Reader-writer is about *access control* to the same shared data — multiple readers can safely access it concurrently since reading doesn't conflict, but a writer needs fully

exclusive access. Producer-consumer's synchronization is buffer-capacity-driven; reader-writer's is about the nature of the operation (read vs write) itself.

Q10. In the dining philosophers problem, what specific scenario causes deadlock, and why?

Answer: If all five philosophers simultaneously pick up their left chopstick before anyone tries for their right, every chopstick ends up held, and every philosopher is now waiting forever for a right chopstick that will never be released — because the philosopher holding it is itself waiting for *its* right chopstick. This is a circular wait: each philosopher holds one resource and waits for another resource held by their neighbor, all the way around the table.

Q11. Give two ways to fix the dining philosophers deadlock.

Answer: (1) Allow only 4 philosophers to attempt picking up chopsticks at once, so at least one chopstick is always free somewhere and the circular wait can't complete. (2) Break the symmetry by having one philosopher pick up chopsticks in the opposite order (right-then-left) — this prevents the perfectly circular dependency that causes the deadlock.

Q12. What is priority inversion? Walk through the classic L/M/H scenario.

Answer: Priority inversion happens when a high-priority process ends up waiting behind a lower-priority one due to a shared lock, even though the OS's scheduler nominally always favors higher priority. Concretely: Low (L) acquires a mutex; High (H) arrives and blocks waiting for that same mutex; Medium (M), which doesn't need the mutex at all, becomes ready and — since M's priority is higher than L's — preempts L and runs to completion first. Only after M finishes does L resume, finish, and release the mutex, letting H finally run. The actual order (M, H, L) means M effectively ran ahead of H, even though H has higher priority than M — priorities were "inverted" for that stretch.

Q13. How does priority inheritance solve priority inversion, and why does it work?

Answer: When a high-priority process is found to be blocked on a lock held by a lower-priority process, the lower-priority process's priority is temporarily raised to match the blocked (higher-priority) process's priority, for as long as it holds the lock. This means the lock-holder can no longer be preempted by any process with priority in between (like M in the example) — it finishes its critical section quickly and releases the lock, after which the high-priority process can proceed essentially without the intermediate process interfering.

Q14. Scenario: A shared counter is incremented by 1000 threads, each running `counter++` a million times, with no synchronization at all. What would you expect to see, and how would you fix it?

Answer: You'd expect the final value of `counter` to be **less than** $1000 \times 1,000,000$ — and the exact shortfall would vary between runs — because `counter++` isn't atomic (load, increment, store), and concurrent threads will regularly overwrite each other's updates due to the race condition described

in Q1. The fix is to protect the increment with mutual exclusion — e.g., wrap it with a binary semaphore/mutex (`wait(mutex); counter++; signal(mutex);`), or use a hardware-backed atomic increment instruction if the language/platform provides one, so each increment is guaranteed to see and build on the true latest value.